Project Executable Files

Date	1 July 2026
Team ID	—
Project Name	Rising Waters – Flood Prediction System
Maximum Marks	3 Marks

This section lists all executable and deployable files for the Rising Waters project so an evaluator can run or deploy the solution.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	NA — flat-file CSV dataset only
5	Environment configuration file (.env.example or similar)	NA — no secrets/API keys required
6	Deployed application URL (if hosted)	No — runs locally; optional IBM Cloud steps documented in README
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	No — not yet containerized
9	Test files and test results	Yes — see Project Testing folder
10	Demo video or walkthrough (if required)	[Add link before final submission]

Step 2: File / Folder Structure

Path	Purpose
app.py	Flask web application entry point
requirements.txt, README.md	Dependencies and project documentation
model/	flood_model.pkl, scaler.pkl, metadata.json — trained artifacts
data/	flood_data.csv — training dataset

Path	Purpose
notebooks/	0_generate_dataset.py, 1_data_visualization.py, 2_preprocessing_and_training.py
templates/	home.html, predict.html, flood_result.html, no_flood_result.html
static/	css/style.css and generated EDA/evaluation images

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	[Not yet deployed — runs at http://127.0.0.1:5000/ locally]
Login Credentials (Demo)	NA — no authentication required
Platform / Hosting Provider	Optional next step: IBM Cloud (Code Engine), per README Section 8
Repository Link	[Add your GitHub/GitLab repository link here before submission]
Demo Video Link	[Add link before final submission]

Step 4: Run Instructions

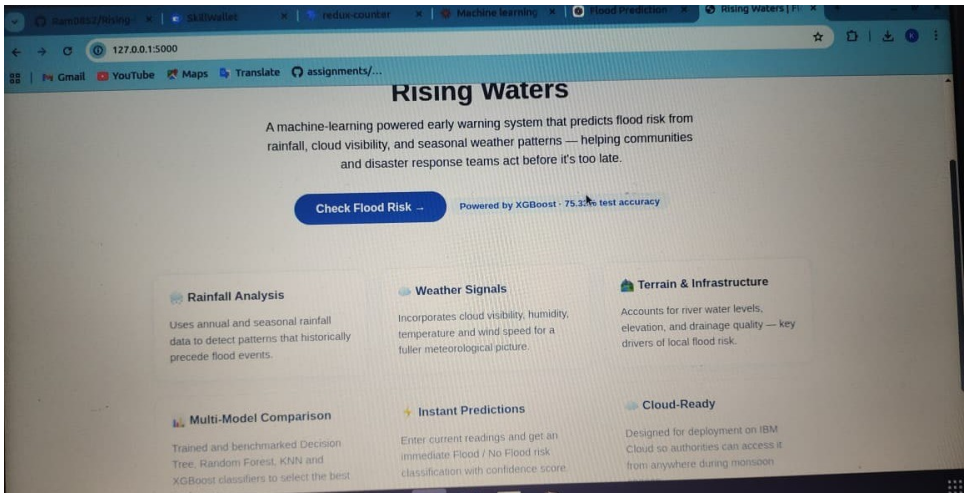
Step	Command
1. Install dependencies	pip install -r requirements.txt
2. Generate the dataset	python notebooks/0_generate_dataset.py
3. (Optional) Run EDA visualizations	python notebooks/1_data_visualization.py
4. Preprocess data & train models	python notebooks/2_preprocessing_and_training.py
5. Run the Flask app	python app.py
6. Open in browser	http://127.0.0.1:5000/

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	xgboost package may be unavailable in some environments	Training script auto-falls-back to sklearn's GradientBoostingClassifier
2	Dataset is simulated, not a real Kaggle dataset	Structure/ranges match a real flood dataset; swap in real data via data/flood_data.csv when available
3	No production WSGI server configured yet	Add gunicorn + PORT env var per README Section 8 before public deployment

Evidence — Application Running Locally

Screenshot of the home page served by app.py at 127.0.0.1:5000, confirming the executable runs successfully end-to-end:


127.0.0.1:5000
Gmail
YouTube
Maps
Translate
assignments/...
Rising Waters
A machine-learning powered early warning system that predicts flood risk from rainfall, cloud visibility, and seasonal weather patterns — helping communities and disaster response teams act before it's too late.
Check Flood Risk →
Rainfall Analysis
Uses annual and seasonal rainfall data to detect patterns that historically precede flood events.
Weather Signals
Incorporates cloud visibility, humidity, temperature and wind speed for a fuller meteorological picture.
Terrain & Infrastructure
Accounts for river water levels, elevation, and drainage quality — key drivers of local flood risk.
Multi-Model Comparison
Trained and benchmarked Decision Tree, Random Forest, KNN and XGBoost classifiers to select the best
Instant Predictions
Enter current readings and get an immediate Flood / No Flood risk classification with confidence score.
Cloud-Ready
Designed for deployment on IBM Cloud so authorities can access it from anywhere during monsoon